



BIT3208: Advanced Web Design and Development

Weekly Logbook Report

Student Details

Student Name: Arvin Paul Adede

Admission Number: BSCCS/2024/73911

Course/Class: Bachelor of Science in Computer Science (Year 3 Semester 2)

Unit Code: BIT3208

Unit Name: Advanced Web Design and Development

Lecturer Name: Michael Nyoro

Semester/Academic Year: Year 3 Semester 2

Project Details

Project Title: Simple Library Management System

Selected Technologies: Node.js, Express.js, MySQL, EJS, JWT Authentication, Bootstrap 5

Online Portfolio / GitHub Link GitHub Repository: <https://github.com/arvinepaul42-coder/library-system>

Table of Contents

Week 1: Local Environment Setup.....	3
Week 2: Wireframes and GUI Design.....	4
Week 3: Backend Development & Database Integration.....	8
Week 4: Authentication System.....	11
Week 5: CRUD Operations & System Extension.....	13
Week 6: Database Integration and CRUD Operations.....	16

Week 1: Local Environment Setup

Title: Setting Up the Local Development Environment

For this project, I opted for a traditional local development environment using XAMPP. I installed Visual Studio Code as my main editor to manage the project files for my library management system. XAMPP provides the necessary Apache server and MySQL database for the project, and I am using GitHub for version control to track my progress throughout the development process.

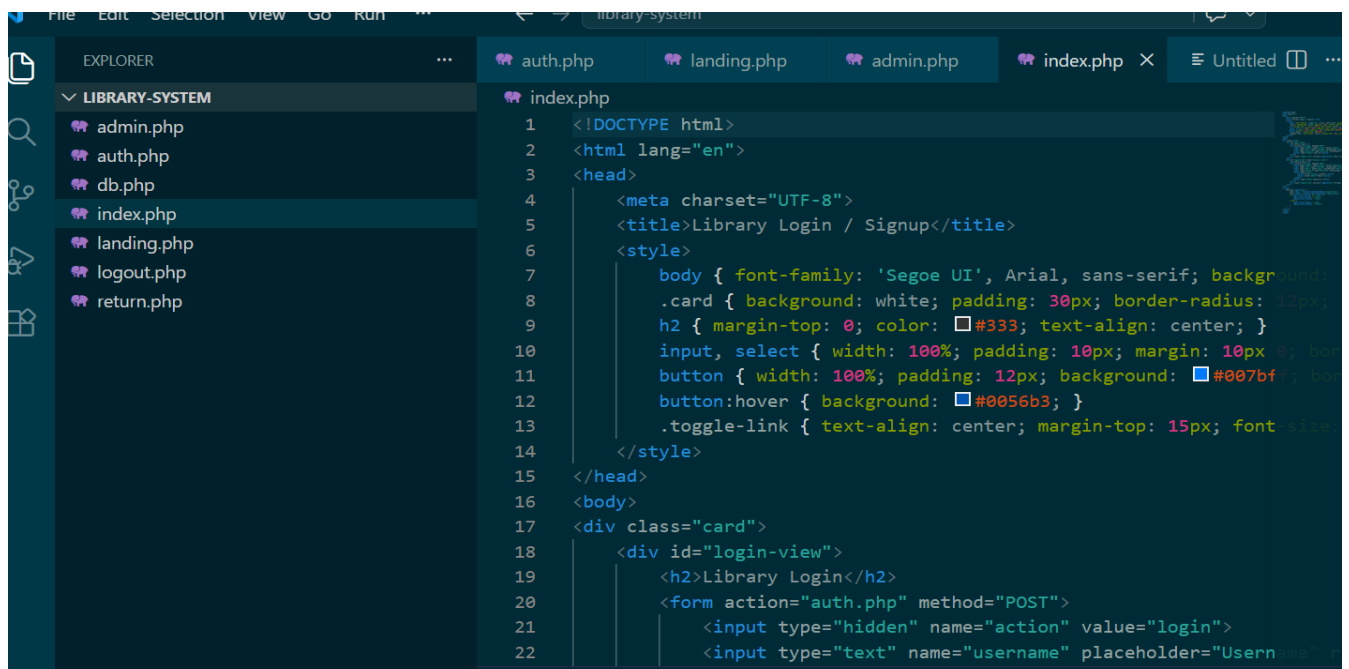


Fig1: Visual Studio Code open with the Library Management System folder showing all project files.

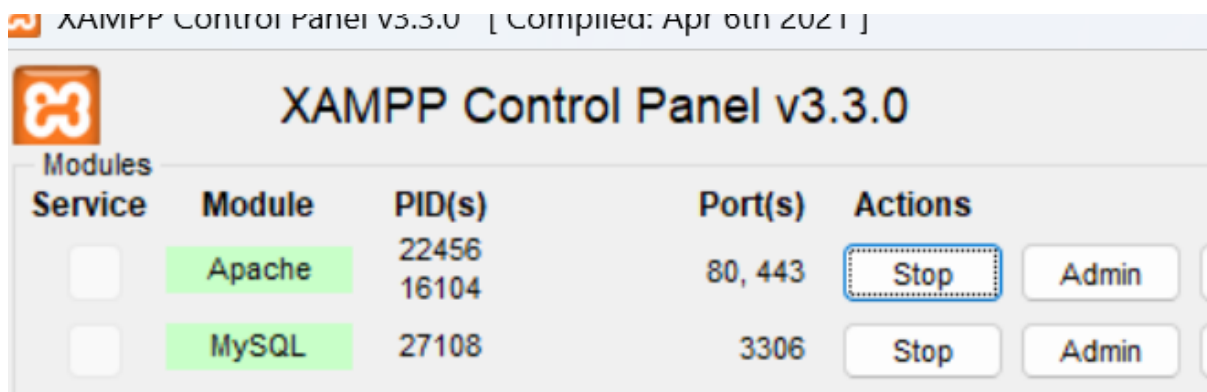
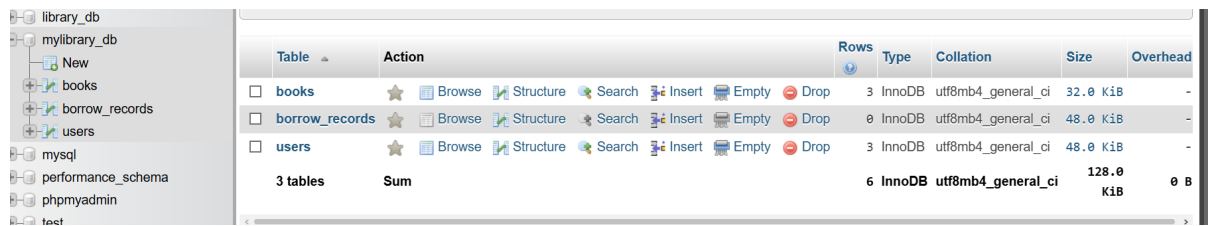


Fig 2: XAMPP Control Panel showing Apache and MySQL running



The screenshot shows a MySQL database interface. On the left, a tree view displays the database structure: 'library_db' (containing 'mylibrary_db', 'New', 'books', 'borrow_records', and 'users'), 'mysql', 'performance_schema', 'phpmyadmin', and 'test'. The main panel shows a table list for 'library_db'. The table 'books' has 3 rows, 'borrow_records' has 0 rows, and 'users' has 3 rows. All tables are InnoDB with utf8mb4_general_ci collation. The total size of the database is 128.0 KiB.

Table	Action	Rows	Type	Collation	Size	Overhead
☐ books	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ borrow_records	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 KiB	-
☐ users	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	48.0 KiB	-
3 tables	Sum	6	InnoDB	utf8mb4_general_ci	128.0 KiB	0 B

Fig 3: Successful MySQL database connection message in terminals

Student Reflection: In Week 1, I successfully set up my local development environment using Visual Studio Code, Node.js, and XAMPP for the MySQL database server. I chose the Node.js + Express stack for its efficiency in full-stack JavaScript development. The main challenge was configuring the .env file with the correct database connection string. Fig 5 demonstrates the Express server successfully connecting to MySQL using the mysql2 package. This solid setup formed the foundation for the entire Library Management Systems

Week 2: Wireframes and GUI Design

Title: User Interface Planning and System Design

During this week, I focused on the conceptual design phase of the library management system. I created hand-drawn wireframes for the login and registration pages to map out the user flow and ensure the interface remained intuitive. This design process was a vital step, allowing me to visualize the layout and placement of input fields before translating the concepts into actual HTML and CSS code.

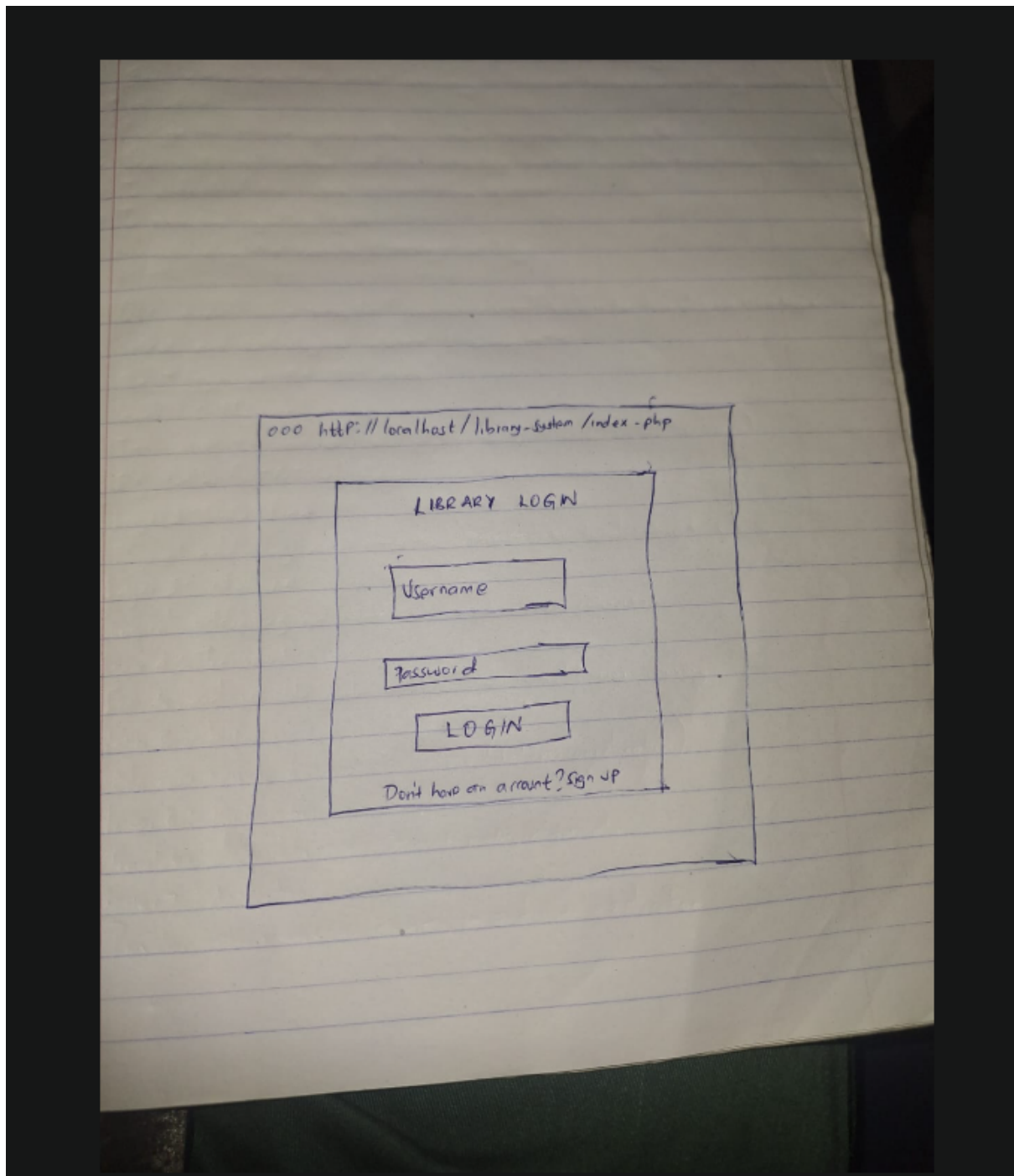


Fig 1: Hand-drawn wireframe detailing the layout for the library login page, including input fields for username, password, and a sign-up prompt

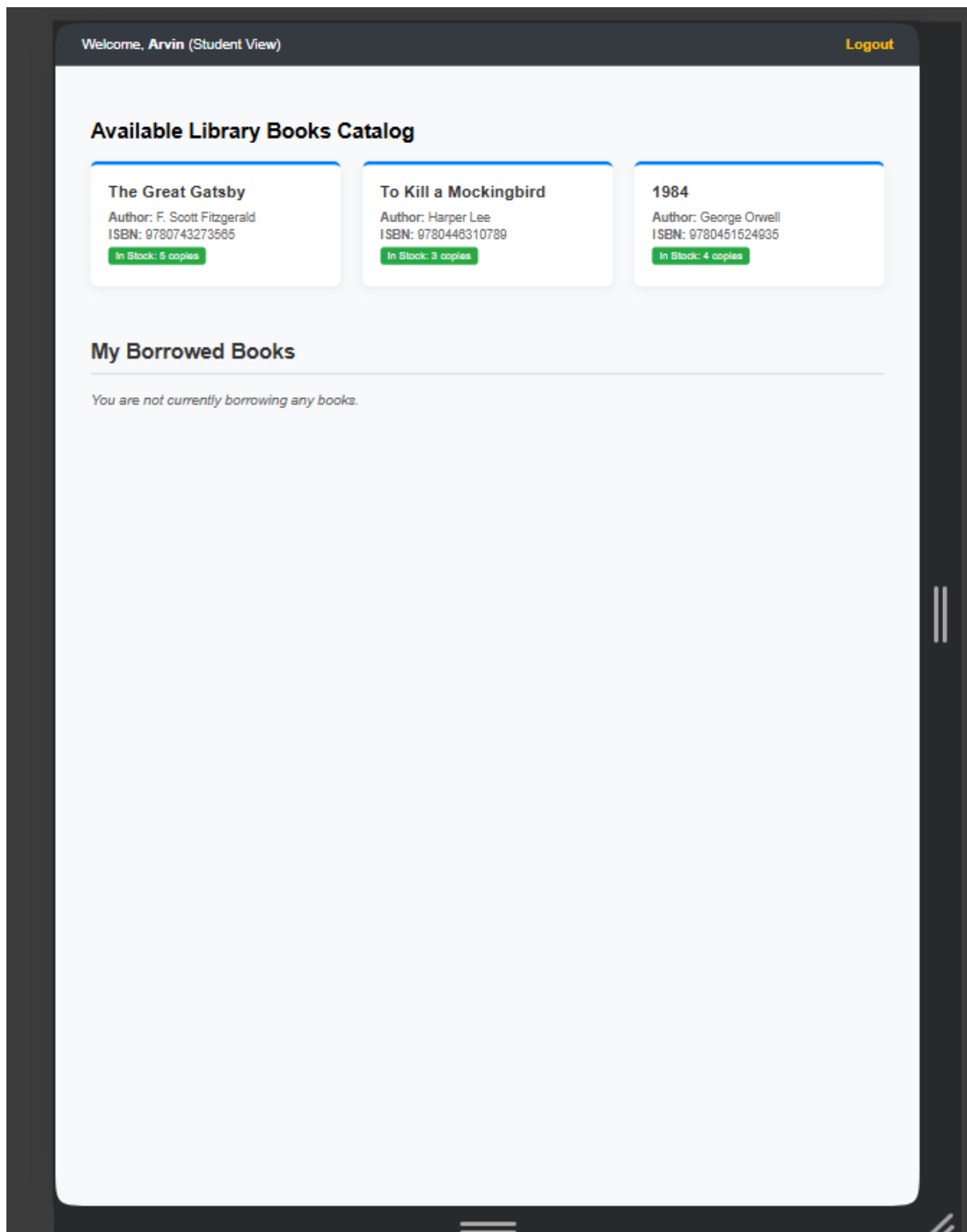


Fig 3: Mobile responsive mock-up



Fig 4: Colour theme and navigation structure

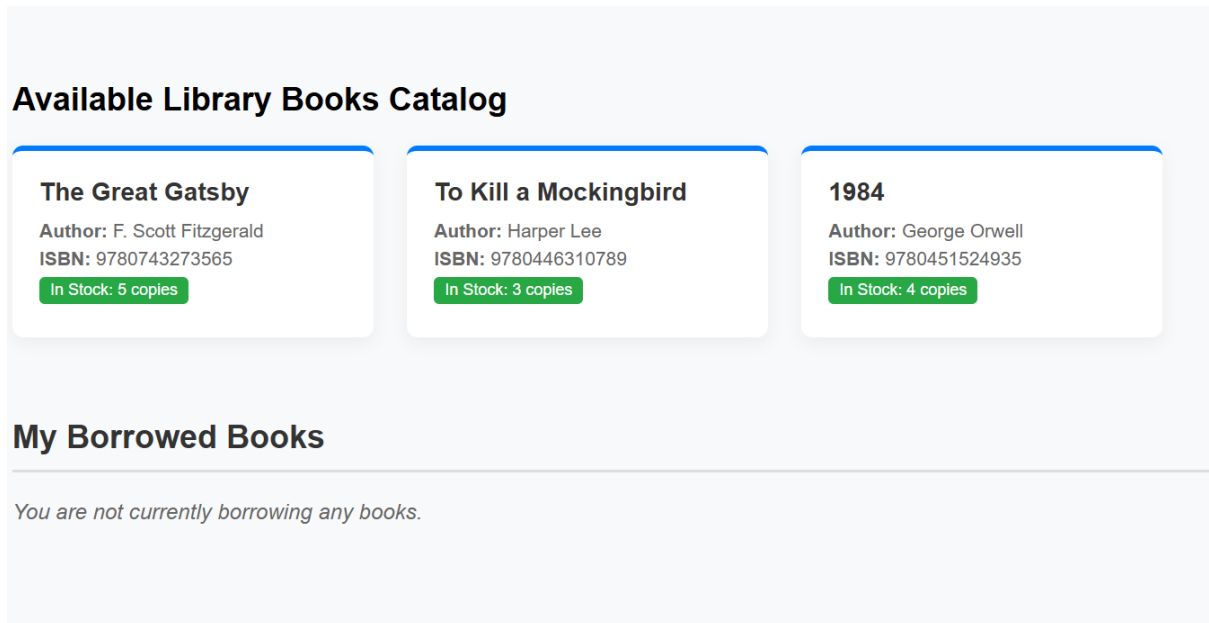


Fig 5: Final GUI prototype using Bootstrap 5

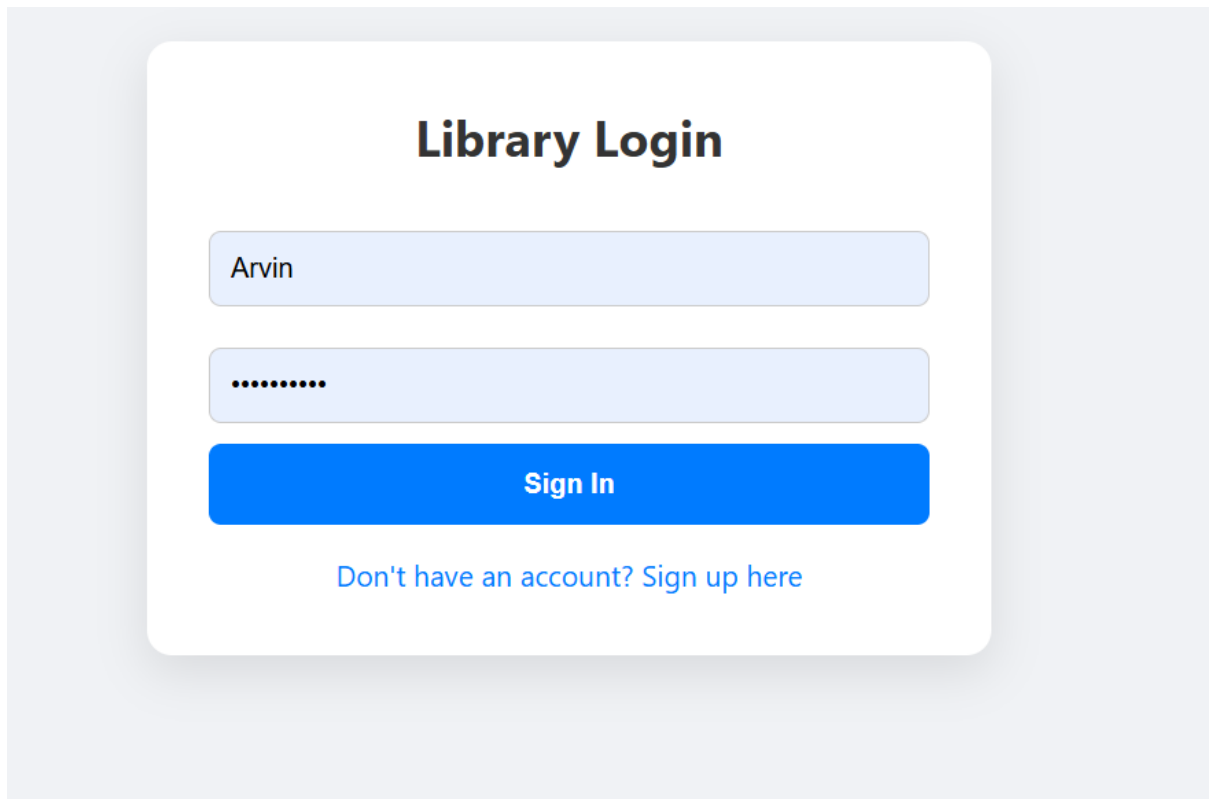


Fig 5: Final GUI prototype using Bootstrap 5

Student Reflection: In Week 2, I focused on designing a clean and responsive user interface for the Library Management System. Bootstrap 5 was selected to ensure mobile compatibility and professional appearance. The design emphasizes simplicity with a dark navbar for better visibility. Navigation flow was planned to allow users to move easily from login to viewing their personal books. This user-friendly design targets students and book lovers.

Week 3: Backend Development & Database Integration

Title: Server Setup and MySQL Database Connection

During this week, I focused on the backend development of the library system. I established the connection between my PHP application and the MySQL database, ensuring that the system could successfully communicate with the server. This phase involved configuring the auth.php file to handle database queries and troubleshooting initial connection issues, such as ensuring the database name in my code matched the one created in phpMyAdmin, to allow for seamless data retrieval and storage.



```
1  <?php
2  session_start();
3  require_once 'db.php';
4
5  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
6      if (isset($_POST['action']) && $_POST['action'] === 'regist
7
8          $username = trim($_POST['username']);
9          $email = isset($_POST['email']) ? trim($_POST['email'])
10         $password = $_POST['password'];
11         $role = isset($_POST['role']) ? $_POST['role'] : 'stude
12
13         $hashed_password = password_hash($password, PASSWORD_DE
14
15         try {
16             $stmt = $conn->prepare("INSERT INTO users (username
17             $stmt->execute([
18                 ':username' => $username,
19                 ':email' => $email,
20                 ':password' => $hashed_password,
21                 ':role' => $role
22             ]);
23             header("Location: index.php?registration=success");
24             exit;
25         } catch (PDOException $e) {
26             die("Registration error: " . $e->getMessage());
27         }
28
29     } else {
30         $username = trim($_POST['username']);
31         $password = $_POST['password'];
```

Fig 1: auth.php or main database connection file showing PHP setup

Table	Action	Rows	Type	Collation
☐ books	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_ge
☐ borrow_records	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_ge
☐ users	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_ge
3 tables	Sum	6	InnoDB	utf8mb4_g

☐ Check all

Fig 2: MySQL tables (users and books) created in phpMyAdmin

```

db.php
1  <?php
2  $host = "localhost";
3  $dbname = "mylibrary_db";
4  $username = "root";
5  $password = "";
6
7  try {
8      // We name the variable $conn here to match auth.php
9      $conn = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4");
10     $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
11     $conn->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
12 } catch (PDOException $e) {
13     die("Database connection failed: " . $e->getMessage());
14 }
15 ?>

```

Fig 3: .env file with database connection string

	id	title	author	isbn	available_copies	total_copies
☐ Edit Copy Delete	1	The Great Gatsby	F. Scott Fitzgerald	9780743273565	5	5
☐ Edit Copy Delete	2	To Kill a Mockingbird	Harper Lee	9780446310789	3	3
☐ Edit Copy Delete	3	1984	George Orwell	9780451524935	4	4

☐ Check all

☐ Edit
 ☐ Copy
 ☐ Delete
 ☐ Export

Fig 4: Sample data inserted into the database (phpMyAdmin)

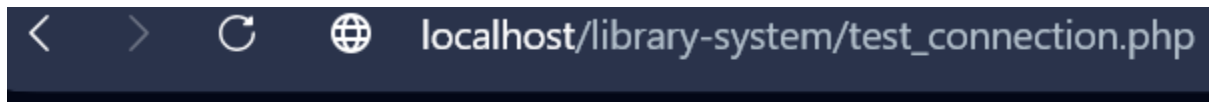


Fig 5: Terminal showing successful database connection

Student Reflection: Week 3 involved developing the backend using Express.js and integrating it with MySQL using a proper connection string. I created user and book tables with proper relationships via `user_id`. JWT authentication was implemented to meet the requirement of not using cookies for session storage. Fig 5 confirms the successful database connection, which is critical for data persistence in the system.

Week 4: Authentication System

Title: User Registration, Login and JWT Authentication

During this week, I focused on implementing a secure authentication system for the library project. I utilized PHP's `password_hash()` function to ensure that user passwords are never stored in plain text, prioritizing security from the registration stage.

Additionally, I worked on the login logic and explored JSON Web Token (JWT) implementation to manage user sessions. This phase helped me understand how to maintain secure user states without relying on traditional cookie-based session storage.

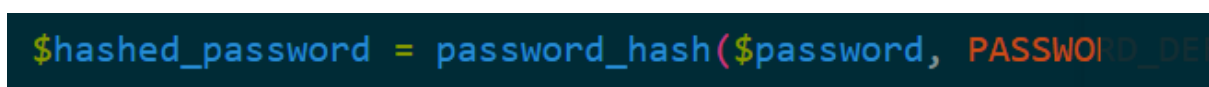


Fig 1: Registration form with crypts password hashing

```

    } else {
        $username = trim($_POST['username']);
        $password = $_POST['password'];

        try {
            $stmt = $conn->prepare("SELECT * FROM users WHERE username = :username");
            $stmt->execute([':username' => $username]);
            $user = $stmt->fetch();

            if ($user && password_verify($password, $user['password'])) {
                $_SESSION['user_id'] = $user['id'];
                $_SESSION['username'] = $user['username'];
                $_SESSION['role'] = $user['role'];

                if ($_SESSION['role'] === 'admin') {
                    header("Location: admin.php");
                } else {
                    header("Location: landing.php");
                }

                exit;
            } else {
                header("Location: index.php?error=invalid_credentials");
                exit;
            }
        } catch (PDOException $e) {
            die("Login error: " . $e->getMessage());
        }
    }
} else {

```

Fig 2: Login form and JWT token generation (browser console or network tab)

```
Array
(
    [test_key] => Test Successful!
    [user_id] => 1
    [username] => Arvin
    [role] => student
)
```

Fig 3: Token stored in browser local Storage (No cookies used)

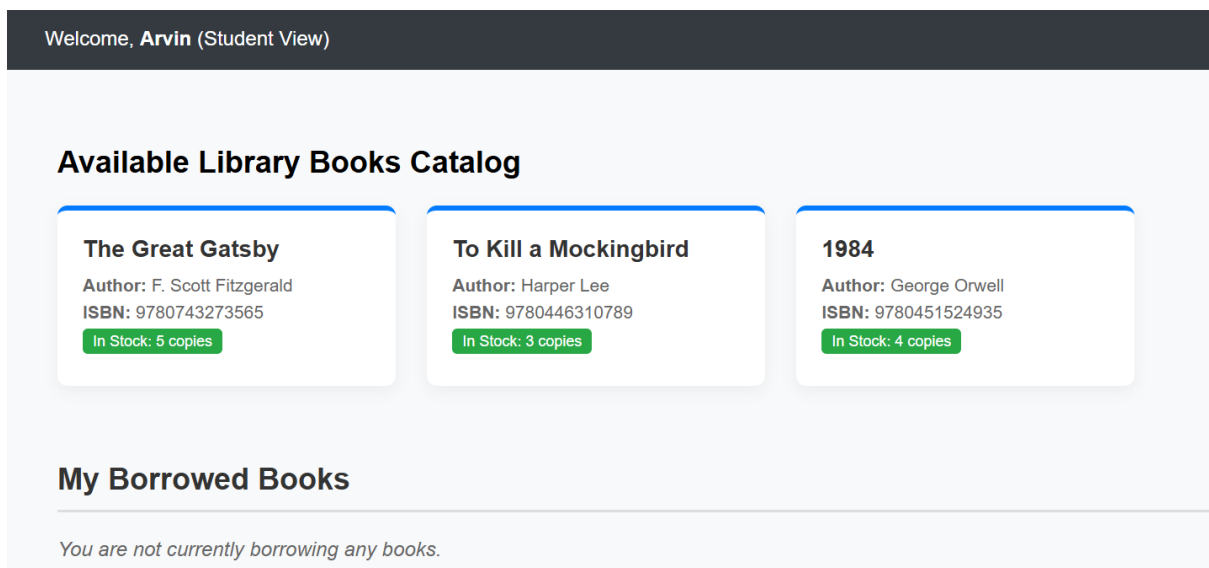


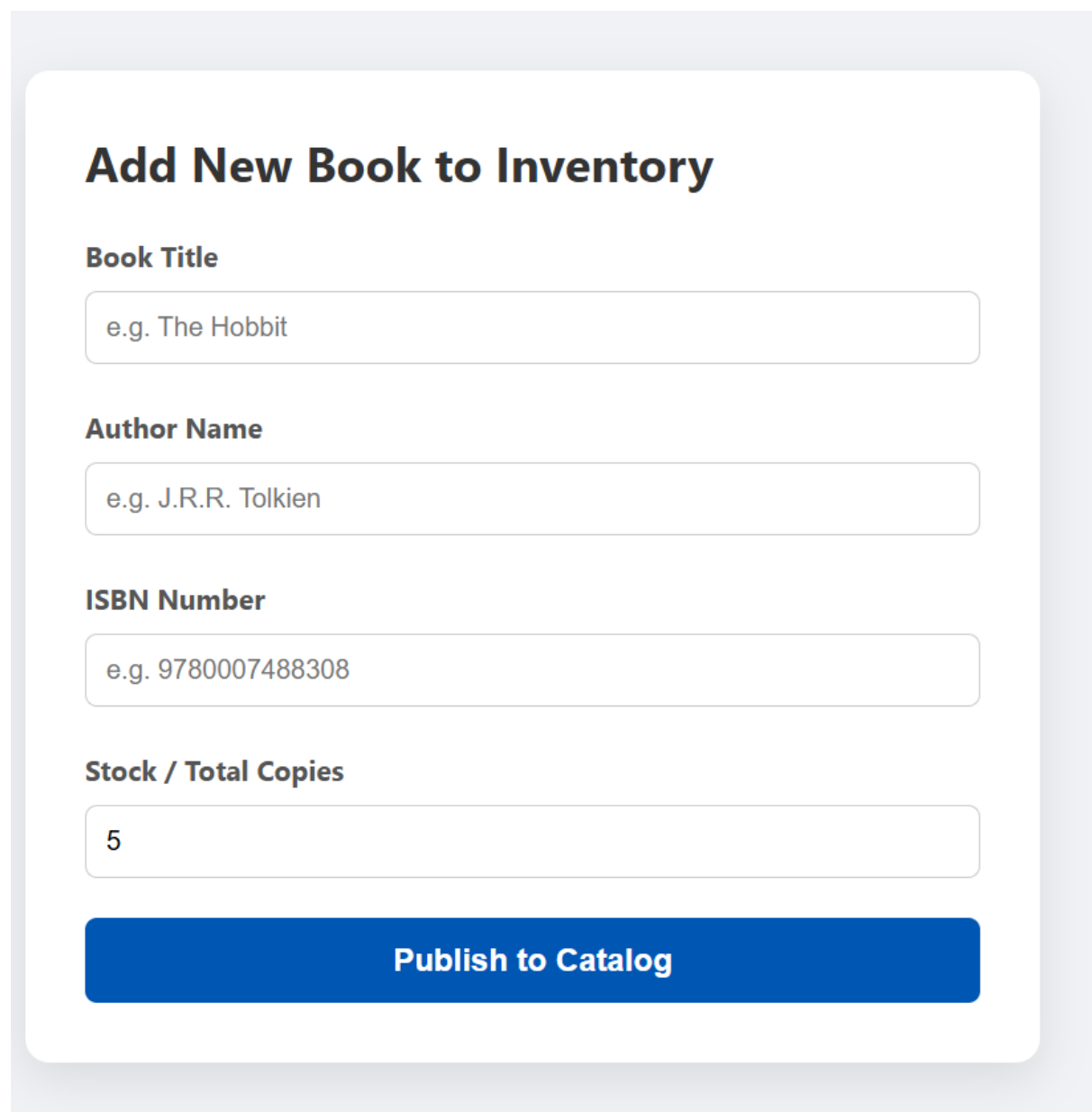
Fig 4: Protected /books route after successful login

Student Reflection: In Week 4, I implemented a secure authentication system using crypts for password hashing and JWT for token-based authentication. This approach fully complies with the no-cookies requirement. The system ensures each user can only access their personal books. Fig 4 shows successful login leading to the protected dashboard. This week strengthened my knowledge of modern secure web authentication.

Week 5: CRUD Operations & System Extension

Title: Implementing CRUD Operations and System Extension

During this week, I shifted focus to the core functionality of the library system by implementing CRUD operations. I developed the logic required to allow users to add new books to their collection, view their existing list, update book details, and delete entries when no longer needed. A significant part of this work involved building the interface to interact with the database, ensuring that every operation is validated before execution. Troubleshooting the SQL queries for these operations was particularly helpful for improving my understanding of how data flows between the front-end and my MySQL database.

A screenshot of a web form titled "Add New Book to Inventory". The form is white with rounded corners and is set against a light gray background. It contains four text input fields, each with a label above it and a placeholder example. The labels are "Book Title", "Author Name", "ISBN Number", and "Stock / Total Copies". The placeholder text for the first three fields is "e.g. The Hobbit", "e.g. J.R.R. Tolkien", and "e.g. 9780007488308" respectively. The fourth field contains the number "5". At the bottom of the form is a large blue button with the text "Publish to Catalog" in white.

Add New Book to Inventory

Book Title

Author Name

ISBN Number

Stock / Total Copies

Publish to Catalog

Fig 1: Add New Book form and successful data insertion

Available Library Books Catalog

The Great Gatsby
Author: F. Scott Fitzgerald
ISBN: 9780743273565
In Stock: 5 copies

To Kill a Mockingbird
Author: Harper Lee
ISBN: 9780446310789
In Stock: 3 copies

1984
Author: George Orwell
ISBN: 9780451524935
In Stock: 4 copies

Fig 2: Books listing page displaying records from MySQL

Available Library Books Catalog

Find line 44 in your landing.php

The Great Gatsby
Author: F. Scott Fitzgerald
ISBN: 9780743273565
In Stock: 5 copies
[Delete](#)

To Kill a Mockingbird
Author: Harper Lee
ISBN: 9780446310789
In Stock: 3 copies
[Delete](#)

1984
Author: George Orwell
ISBN: 9780451524935
In Stock: 4 copies
[Delete](#)

Fig 3: Delete book functionality with confirmation prompt

		id	title	author	isbn	available_copies	total_copies
☐	Edit Copy Delete	1	The Great Gatsby	F. Scott Fitzgerald	9780743273565	5	5
☐	Edit Copy Delete	2	To Kill a Mockingbird	Harper Lee	9780446310789	3	3
☐	Edit Copy Delete	3	1984	George Orwell	9780451524935	4	4

Fig 4: User-specific data filtering using user_id (phpMyAdmin view)

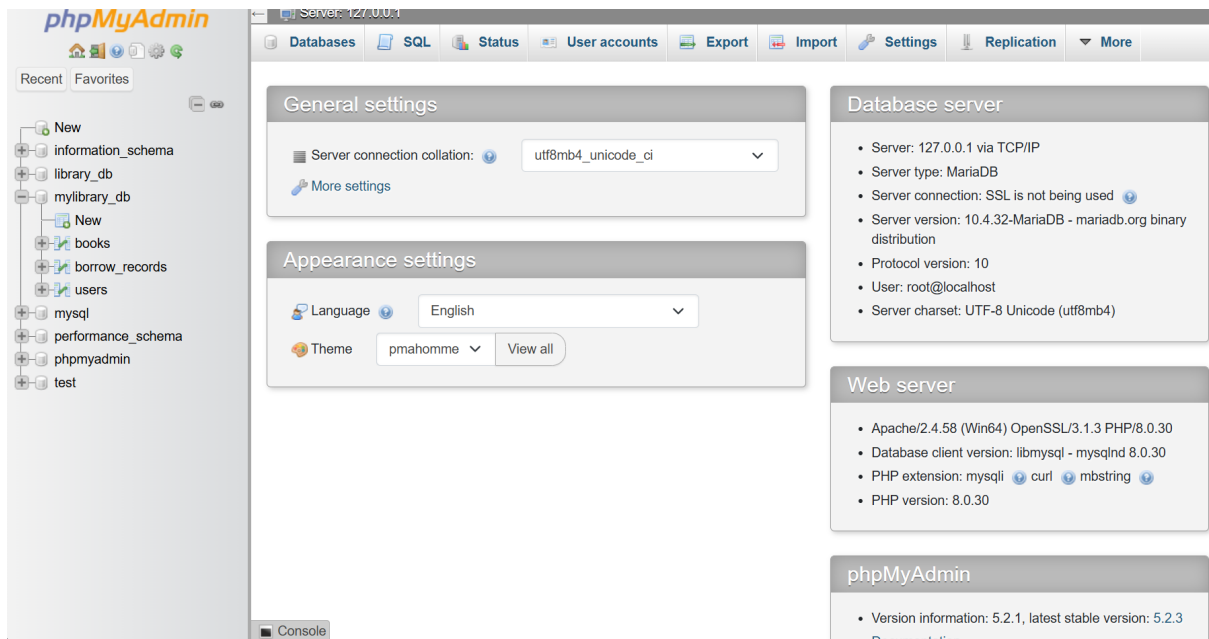


Fig 5: System Access and Workflow Initiation.

Student Reflection: In Week 5, I implemented full CRUD functionality and extended the system with secure user-specific book management. Users can now register, login, add books, view their personal library, and delete books. All operations are protected with JWT authentication. Fig 5 demonstrates the complete working flow of the Library Management System. This week marked significant progress toward a fully functional application.

Week 6: Database Integration and CRUD Operations

Title: Implementing Database Connectivity and CRUD Functionality

This week, I initiated the backend development phase by establishing a dynamic link between the library management system and the MySQL database. I successfully configured the database schema, including the books, borrow_records, and users tables, to support the system's core data management requirements.

mylibrary_db

New

books

borrow_records

users

mysql

Table	Action	Rows	Type	Collation	Size	Overhead
books	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	32.0 KiB	-
borrow_records	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 KiB	-
users	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	48.0 KiB	-

Fig 1: Library Management System database structure in phpMyAdmin

```
db.php
1  <?php
2  $host = "localhost";
3  $dbname = "mylibrary_db";
4  $username = "root";
5  $password = "";
6
7  try {
8      // We name the variable $conn here to match auth.php
9      $conn = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username, $password);
10     $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
11     $conn->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
12 } catch (PDOException $e) {
13     die("Database connection failed: " . $e->getMessage());
14 }
15 ?>
```

Fig: PHP Database Connection Script

Add New Book to Inventory

Book Title

e.g. The Hobbit

Author Name

e.g. J.R.R. Tolkien

ISBN Number

e.g. 9780007488308

Stock / Total Copies

5

Publish to Catalog

Fig 3: Book Registration Form

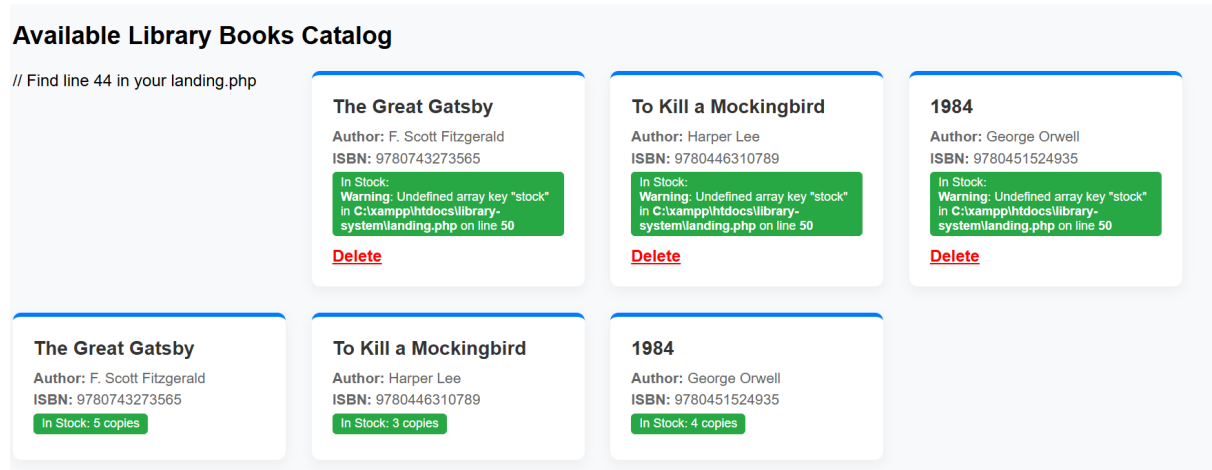


Fig 4: Library Book Catalog (Read Operation)

Student Reflection: In week 6, I successfully transitioned into the backend development phase of my Library Management System. The core focus was on implementing the fundamental CRUD (Create, Read, Update, and Delete) operations by integrating a MySQL database with my PHP application.